Tutoriumsblatt 0

Deine Entwicklungsumgebung

Bearbeitungszeit etwa 90 Minuten · 40 Punkte · Lösungen im Anhang ab Seite 6

Nach diesem Blatt kannst du

- Versionsangaben, Lockfile, Kodierungen und Prüfsummen von Hand auswerten,
- Programme und Aufrufe vorhersagen und Code im Review beurteilen.

So arbeitest du mit dem Blatt

Du brauchst keinen Rechner, nur Papier, Stift und Taschenrechner. Jedes Programm läuft für sich in Python 3.13; schreib Ausgaben Zeile für Zeile so auf, wie Python sie druckt, stdout und stderr getrennt; Bytes hexadezimal mit zwei Ziffern je Byte (`C3 A4`), Positionen ab 0. Hängst du, rechne mit den Werten unter „Zur Kontrolle“ weiter. „Ausgedacht“ heißt fiktiv; Befehle stehen in der Form für macOS und Linux. Zum Notebook: Aufgabe 1 gehört zum Schritt „Auf deinem Laptop“; Aufgabe 2 zu TODO 2 und 9; Aufgabe 3 zu 4 bis 7 und 11; Aufgabe 4 zu 10 und 12.

Aufgabe 1 Versionsangaben und Lockfile

9 Punkte · etwa 18 Min.

Bezug: Modul 0, Lerneinheiten 2 (Python-Versionen, virtuelle Umgebungen und uv) und 7 (In der IDE arbeiten und der Nachweis)

Die Werks-IT in Illinois stellt ihr Berichtswerkzeug als Paket `shiftreport` bereit (ausgedacht). Im Paketindex liegen `1.8.2`, `1.9.0`, `1.9.7`, `1.10.0`, `1.10.3`, `2.0.0rc1`, `2.0.0` und `2.1.1`. uv wählt die neueste erlaubte Version, die Vorabversion `2.0.0rc1` nur, wenn eine Angabe selbst eine Vorabversion nennt. Bei `~=` darf sich nur die letzte angegebene Stelle erhöhen, bei `==` zählen fehlende Stellen als 0.

(a) Gib für jede Angabe die erlaubten Versionen an und die Version, die uv wählt: (1) `>=1.9`, (2) `>=1.9,<2`, (3) `~=1.9.0`, (4) `~=1.9`, (5) `==1.10`. Ein Kollege sucht die neueste Version von Hand mit `max(verfuegbar)` über die Liste der Texte `"1.8.2"`, `"1.9.0"`, `"1.9.7"`, `"1.10.0"`, `"1.10.3"`. Was liefert das? Zur Kontrolle: Angabe (3) erlaubt genau zwei Versionen. [4 P]

Kim stellte `pyproject.toml` und `uv.lock` ins Repository, als `1.10.0` die neueste Version war. Jeder Fall unten beginnt mit einem frischen Klon bei Lea.

```
# pyproject.toml, Ausschnitt
[project]
name = "nordwerk"
requires-python = ">=3.13,<3.14"
dependencies = ["shiftreport>=1.9,<2"]
```

```
# uv.lock, Ausschnitt
[[package]]
name = "shiftreport"
version = "1.10.0"
```

(b) Passt das Lockfile nicht zu `pyproject.toml`, schreibt `uv sync` es neu, `uv sync --locked` nie. `uv lock` und `uv sync` behalten gesperrte Versionen, solange die Angabe sie erlaubt.

Welche Version hat Lea danach, endet der Befehl mit 0, ändert sich `uv.lock`? Zur Kontrolle: In genau einem Fall bekommt Lea eine andere Version als die in Kims Lockfile. [3 P]

Fall	Stand im Repository	Befehl bei Lea
1	wie oben	`uv sync --locked`
2	wie oben	`uv sync`
3	Angabe geändert in `shiftreport>=1.9,<3`, danach `uv lock`	`uv sync --locked`
4	Angabe geändert in `shiftreport>=1.10.3,<2`, ohne `uv lock`	`uv sync`

(c) Welche Dateien gehören in Kims Repository? Begründe kurz: `nordwerk.toml`, `.gitignore`, `.vscode/launch.json`, `notebooks/W00_Entwicklungsumgebung_Teilnehmende.ipynb`, `src/nordwerk/__pycache__/`, `daten/roh/IL_shift_report_07132026.csv`, `daten/aufbereitet/schichten.csv`. [2 P]

Aufgabe 2 Bytes, Kodierung und Prüfsumme

10 Punkte · etwa 22 Min.

Bezug: Modul 0, Lerneinheiten 1 (Bei mir läuft's: warum Umgebungen zählen) und 4 (Robuste Funktionen: Pfade, Kodierung, Ausnahmen und Logging)

Für die Zeichen links ist das Byte in Windows-1252 gleich dem Codepunkt; bis `7F` ist auch UTF-8 dasselbe eine Byte (A bis Z `41` bis `5A`, a bis z `61` bis `7A`, Komma `2C`). In UTF-8 (Mitte) füllen die Bits des Codepunkts die Stellen `x` von rechts, links mit 0 aufgefüllt; `10xxxxxx` steht nur als Folgebyte, Startbytes ab `F5` gibt es nicht. Rechts liest Windows-1252 einige Bytes.

Zeichen	Codepunkt
G e h	`47 65 68`
r s u	`72 73 75`
ä ö	`E4 F6`
ß ü	`DF FC`

Codepunkt	Bytes in UTF-8
bis `7F`	`0xxxxxxx`
`80` bis `7FF`	`110xxxxx 10xxxxxx`
`800` bis `FFFF`	`1110xxxx 10xxxxxx 10xxxxxx`

Bytes	gelesen
`C3 EF`	Ã ï
`A4 B6`	¤ ¶
`BB BC`	» ¼
`BF 9F`	¿ Ÿ

(a) Schreib „Gehäuse“ (Linie 2 im Stammwerk ist die Gehäusefertigung) und „Größe“ (ein Spaltenname, den die Werks-IT plant, ausgedacht) als Bytes in UTF-8 und in Windows-1252, jeweils mit der Zahl der Bytes. Leite die UTF-8-Bytes von ö und ß aus den Bits her. Zur Kontrolle: ä ist in UTF-8 `C3 A4`. [3 P]

(b) Seit dem MES-Update am 1. September 2026 schreibt das Stammwerk UTF-8, ein altes Makro liest aber Windows-1252. Was zeigt es statt „Früh“ und „Gehäuse“? Wo und warum bricht umgekehrt `decode("utf-8")` bei „Gehäuse“ in Windows-1252 mit „invalid continuation byte“ ab, beim „ü“ aus „Früh“ aber mit „invalid start byte“? Woran erkennt ein Programm am Dateianfang, dass eine Illinois-Datei UTF-8 ist? [3 P]

(c) Ein Auszug (ausgedacht) mit drei Zeilen, jede mit Zeilenende (rechts). Berechne die Größe der Datei in Byte (1) in UTF-8 mit `\n` wie `schreibe_csv`, (2) in UTF-8 mit `\r\n`, (3) unter Windows mit deutscher Spracheinstellung mit `open("auszug.csv", "w")` geschrieben, (4) als UTF-8 mit BOM und `\n`.

```
werk,linie,schicht,temp_c
Stammwerk,2,Spät,22.4
Illinois,B,Früh,28.9
```

Warum sind alle vier Prüfsummen (SHA-256) verschieden, obwohl ein Programm, das Kodierung und Zeilenende richtig erkennt, viermal dieselbe Tabelle daraus macht, und warum unterscheiden sich (2) und (4) trotz gleicher Größe? Die Prüfsumme des Importers sind die ersten 12 Hexadezimalzeichen von SHA-256: Wie viele Bit sind das, und wie wahrscheinlich hat eine andere Datei zufällig dieselbe? Zur Kontrolle: Die erste Zeile hat 25 Zeichen. [4 P]

Aufgabe 3 Ablauf vorhersagen

11 Punkte · etwa 26 Min.

Bezug: Modul 0, Lerneinheiten 4 (Robuste Funktionen: Pfade, Kodierung, Ausnahmen und Logging) und 5 (Generatoren, Scope und Typen)

(a) Die Zellen kommen aus der Summenzeile `TOTAL,,,2873,2552,66,,` der Illinois-Datei vom 13. Juli. Was druckt das Programm rechts? Was ändert sich, wenn `from e` fehlt? [2 P]

```
class ZeilenFehler(ValueError): """Zeile unbrauchbar"""
def stueck(zelle):
    try:
        wert = int(zelle)
    except ValueError as e:
        raise ZeilenFehler(f"unlesbar: {zelle!r}") from e
    else:
        return wert
    finally:
        print("fertig", repr(zelle))
for zelle in ["2552", "", "66"]:
    try:
        print("Summe", stueck(zelle) + 1)
    except ValueError as e:
        print(type(e).__name__, type(e.__cause__).__name__, e)
```

(b) Drei Hallentemperaturen einer Illinois-Linie an einem Tag in Grad Fahrenheit (ausgedacht). Was druckt das Programm rechts? Rechne die Umrechnung für 84,8 °F von Hand. Zur Kontrolle: Das Programm druckt 13 Zeilen. [2 P]

```
def celsius(werte_f):
    print("Start")
    for f in werte_f:
        print("vor", f)
        yield round((f - 32) * 5 / 9, 1)
        print("nach", f)
    print("Ende")
gen = celsius([83.0, 84.8, 88.4])
print("angelegt")
print("erster", next(gen))
for c in gen:
    print("Schleife", c)
print("Rest", list(gen))
```

(c) Was druckt das Programm rechts, und was passiert ohne `nonlocal neu`? [2 P]

```
werk, gelesen = "Stammwerk", 0
def lauf(namen):
    global gelesen
    werk, neu = "Illinois", 0
    def zaehle(name):
        nonlocal neu
        if name.endswith(".csv"):
            neu += 1
        return f"{werk}/{name}"
    pfade = [zaehle(n) for n in namen]
    gelesen += neu
    return pfade, neu
print(lauf(["IL_shift_report_07132026.csv", "liesmich.txt"]))
print(lauf(["stammwerk_2026-07-13.csv"])[1], werk, gelesen)
```

(d) Was druckt das Programm rechts? [1 P]

```
STUFE = "WARNING"
def zaehle(grund, zaehler={}, stufe=STUFE):
    zaehler[grund] = zaehler.get(grund, 0) + 1
    return stufe, zaehler
STUFE = "DEBUG"
print(zaehle("summenzeile"))
print(zaehle("unplausibel", {}))
print(zaehle("summenzeile"))
```

(e) Das Programm unten steht in `melde.py`. Was erscheint auf stdout, was auf stderr? Was steht nach `python melde.py > bericht.txt` in der Datei und was im Terminal? Was ändert sich mit `log.setLevel(logging.WARNING)` in der vierten Zeile? [2 P]

```
import logging
import sys
log = logging.getLogger("nordwerk.importer")
log.setLevel(logging.INFO)
aus = logging.StreamHandler(sys.stdout)
aus.setFormatter(logging.Formatter("%(levelname)s %(message)s"))
fehler = logging.StreamHandler()
fehler.setLevel(logging.ERROR)
fehler.setFormatter(logging.Formatter("%(name)s: %(message)s"))
log.addHandler(aus)
log.addHandler(fehler)
log.debug("%d Dateien gefunden", 72)
log.info("%s: %d Dubletten", "stammwerk_2026-08-06.csv", 12)
log.warning("%s: %d ohne Temperatur", "stammwerk_2026-07-14.csv", 2)
log.error("%s: Datei ist leer", "stammwerk_2026-07-22.csv")
```

(f) Berechne auf eine Nachkommastelle das Mittel der gerundeten Werte aus (b) und das Mittel der Fahrenheit-Werte, erst umgerechnet und dann gerundet. Zeige, dass Umrechnen und Mitteln vertauschbar sind, Runden und Mitteln nicht. Wie weit weicht ein Mittel gerundeter Werte vor dem letzten Runden höchstens vom Mittel der ungerundeten Werte ab? Zur Kontrolle: Die Fahrenheit-Werte haben das Mittel 85,4 °F. [2 P]

Aufgabe 4 Konfiguration, Kommandozeile und Review

10 Punkte · etwa 22 Min.

Bezug: Modul 0, Lerneinheiten 3 (Ein eigenes Paket mit Kommandozeilenbefehl) und 6 (Konfiguration und Geheimnisse)

`nordwerk import` bestimmt `eingang`, `ausgang` und `log_level` wie im Skript: Argument vor Umgebungsvariable (`NORDWERK_EINGANG` usw.) vor `[import]` in `nordwerk.toml` vor Voreinstellung (`daten/roh`, `daten/aufbereitet`, `WARNING`). Rechts Leas Dateien; vorher ist keine Variable `NORDWERK_...` gesetzt.

```
# nordwerk.toml
[import]
eingang = "daten/roh"
ausgang = "export"
# .env
NORDWERK_AUSGANG=daten/test
NORDWERK_LOG_LEVEL=INFO
```

(a) Gib für jeden Aufruf `eingang`, `ausgang` und `log_level` an und woher jeder Wert kommt. Zur Kontrolle: In genau zwei Fällen ist `ausgang` gleich `daten/test`. [3 P]

Fall	Aufruf
1	`uv run nordwerk import`
2	`uv run --env-file .env nordwerk import`
3	`uv run --env-file .env nordwerk import daten/juli --log-level DEBUG`
4	`NORDWERK_EINGANG=/mnt/werk/roh uv run nordwerk import --aus ergebnis`
5	wie Fall 1, aber in `nordwerk.toml` steht `[Import]` statt `[import]`

Rechts steht `zaehle.py` einer Kollegin im Projektordner.

(b) Was erscheint bei den Aufrufen unten auf stdout, was (kurz) auf stderr, und welcher Exit-Code folgt? Die Illinois-Datei ist eine Woche ohne Feiertag, die Dateien vom 21. Juli und 15. September sind volle Produktionstage. Zur Kontrolle: Zwei Aufrufe enden mit 0. [3 P]

```python
import argparse
import sys
from pathlib import Path
def main(argv=None):
    p = argparse.ArgumentParser(prog="zaehle")
    p.add_argument("datei")
    p.add_argument("--trenn", default=",")
    args = p.parse_args(argv)
    daten = Path(args.datei).read_bytes()
    if not daten:
        print(args.datei, "ist leer", file=sys.stderr)
        return 1
    kopf, *rest = daten.decode("utf-8").splitlines()
    spalten = kopf.count(args.trenn) + 1
    print(len(rest), "Zeilen,", spalten, "Spalten")
    return 0
raise SystemExit(main())
```

Nr.	Aufruf im Projektordner
1	`uv run python zaehle.py daten/roh/IL_shift_report_07132026.csv`
2	`uv run python zaehle.py daten/roh/stammwerk_2026-07-21.csv --trenn ";"`
3	`uv run python zaehle.py daten/roh/stammwerk_2026-09-15.csv`
4	`uv run python zaehle.py daten/roh/stammwerk_2026-07-22.csv 2>/dev/null`
5	`uv run python zaehle.py --trenn ";"`

Rechts Kims Vorschlag für einen Monatsbericht.

(c) Nenne fünf Probleme in Kims Vorschlag mit Folge und Lösung im Paket `nordwerk`. [2 P]

```python
import glob
TOOLBOX_KEY = "tbx-7f3a91c2"
EINGANG = "C:\\Users\\kim\\Desktop\\roh"
def lies_stammwerk():
    zeilen = []
    for name in glob.glob(EINGANG + "\\stammwerk_*.csv"):
        try:
            text = open(name).read()
        except:
            pass
        print("gelesen:", name)
        zeilen += text.splitlines()[1:]
    return zeilen
```

(d) Unter Linux liest `open` UTF-8. (1) Was liefert `lies_stammwerk()` dort unverändert, was wird ausgegeben? (2) Lea setzt `EINGANG = "daten/roh"` und im Muster `"/"` statt `"\\"`. Im Ordner liegen nur die Dateien vom 31. August und 1. September (je zwölf Schichten), `glob` liefert die vom 1. September zuerst. Was wird ausgegeben, wie viele Zeilen kommen zurück, was steckt darin? (3) Und andersherum? [2 P]

Anhang: Lösungen

Erst selbst rechnen, dann vergleichen. Die Lösungen zeigen den Rechenweg und typische Fehler.

Lösung zu Aufgabe 1 Versionsangaben und Lockfile

(a) Versionen werden Teil für Teil als Zahlen verglichen, `1.10.0` liegt also nach `1.9.7`.

Angabe	bedeutet	erlaubt	uv wählt
(1) `>=1.9`	ab `1.9`	`1.9.0`, `1.9.7`, `1.10.0`, `1.10.3`, `2.0.0`, `2.1.1`	`2.1.1`
(2) `>=1.9,<2`	ab `1.9`, unter `2`	`1.9.0`, `1.9.7`, `1.10.0`, `1.10.3`	`1.10.3`
(3) `~=1.9.0`	`>=1.9.0`, gleiches `1.9`	`1.9.0`, `1.9.7`	`1.9.7`
(4) `~=1.9`	`>=1.9`, gleiche `1`	`1.9.0`, `1.9.7`, `1.10.0`, `1.10.3`	`1.10.3`
(5) `==1.10`	genau `1.10.0`	`1.10.0`	`1.10.0`

`2.0.0rc1` fehlt überall, keine Angabe nennt eine Vorabversion. (2) und (4) erlauben dasselbe, (4) sagt es kürzer. Als Text vergleicht Python Zeichen für Zeichen, und „9“ kommt nach „1“: `max(verfuegbar)` liefert `1.9.7`. Erst ein Vergleich Teil für Teil als Zahlen findet `1.10.3`, und für Vorabversionen wie `2.0.0rc1` braucht es die Regeln, die uv kennt.

(b)

Fall	Exit-Code	`shiftreport`	`uv.lock`
1	0	`1.10.0`	unverändert
2	0	`1.10.0`	unverändert
3	0	`1.10.0`	unverändert, Kims neues Lockfile sperrt weiter `1.10.0`
4	0	`1.10.3`	neu geschrieben, jetzt `1.10.3`

In den Fällen 1 und 2 passt das Lockfile zur Angabe; beide installieren `1.10.0` wie bei Kim, obwohl `1.10.3` erlaubt wäre. Neuere Versionen kommen erst, wenn jemand bewusst aktualisiert, etwa mit `uv lock --upgrade-package shiftreport`. In Fall 3 hat Kims `uv lock` die gesperrte Version behalten, weil `<3` sie weiter erlaubt: Lea bekommt wieder `1.10.0`, obwohl jetzt sogar `2.1.1` erlaubt wäre. In Fall 4 erlaubt die Angabe `1.10.0` nicht mehr; uv löst neu auf und nimmt die einzige erlaubte Version `1.10.3`. Leas `uv.lock` weicht nun vom Repository ab, mit `--locked` hätte der Befehl mit einem Exit-Code ungleich 0 abgebrochen. Richtig wäre: Kim führt `uv lock` aus und stellt `pyproject.toml` und `uv.lock` gemeinsam ins Repository.

(c) Hinein gehören vier Dateien: `nordwerk.toml` (gemeinsame Voreinstellungen), `.gitignore` (sagt Git, was draußen bleibt), `.vscode/launch.json` (Start im Debugger) und das Notebook (die Übung, aus der der Code stammt). Sie sind auf jedem Laptop gleich. Nicht hinein gehören `__pycache__/` (erzeugt Python beim Import), die Rohdaten (kommen aus dem Zip) und `schichten.csv` (Code und Rohdaten stellen es wieder her, die Prüfsumme belegt, dass es gleich ist). `__pycache__/` steht schon in der `.gitignore` von `uv init`, `daten/` trägst du nach.

Typischer Fehler

In (a) die Versionen als Text vergleichen und bei (2) `1.9.7` für die neueste halten. In (b) bei Fall 2 oder 3 annehmen, uv hole die neueste erlaubte Version. Solange die Angabe die gesperrte Version erlaubt, bleibt sie; genau dafür gibt es das Lockfile.

Bewertung: (a) erlaubte Versionen 2 P, Wahl von uv 1 P, Textvergleich 1 P. (b) Fälle 1 und 2 1 P, Fall 3 1 P, Fall 4 1 P. (c) Entscheidung 1 P, Begründungen 1 P.

Lösung zu Aufgabe 2 Bytes, Kodierung und Prüfsumme

(a) ö hat den Codepunkt `F6` = 1111 0110, mit 11 Bit 000 1111 0110. Aufgeteilt in 5 und 6 Bit: 00011 und 110110, also 110 00011 = `C3` und 10 110110 = `B6`. ß hat `DF` = 1101 1111, aufgeteilt 00011 und 011111, also `C3` und 10 011111 = `9F`.

Wort	UTF-8	Windows-1252
Gehäuse	`47 65 68 C3 A4 75 73 65`, 8 Byte	`47 65 68 E4 75 73 65`, 7 Byte
Größe	`47 72 C3 B6 C3 9F 65`, 7 Byte	`47 72 F6 DF 65`, 5 Byte

Jeder Umlaut und das ß kosten in UTF-8 zwei Bytes, in Windows-1252 eines.

(b) Windows-1252 liest jedes Byte einzeln als ein Zeichen (nur fünf Bytes wie `81` sind dort nicht belegt): `46 72 C3 BC 68` wird „FrÃ¼h“, `47 65 68 C3 A4 75 73 65` wird „GehÃ¤use“. Aus jedem Umlaut werden zwei Zeichen, ohne Fehlermeldung. Umgekehrt kündigt `E4` = 1110 0100 an Position 3 ein Zeichen mit drei Bytes an, das nächste Byte müsste `10xxxxxx` sein. Es ist `75` = 0111 0101: Abbruch bei `0xe4`, Position 3 mit „invalid continuation byte“. `FC` = 1111 1100 passt zu keinem Startbyte, deshalb dort „invalid start byte“. Eine Illinois-Datei beginnt mit der BOM `EF BB BF`; in Windows-1252 wären das die Zeichen „ï»¿“ vor `Date`, als Anfang einer Datei sind sie das Zeichen für UTF-8 mit BOM.

(c) Die Zeilen haben 25, 21 und 20 Zeichen, zusammen 66, davon 2 Umlaute (ä, ü).

Fall	Rechnung	Byte	Prüfsumme
(1) UTF-8, `\n`	$66 + 2 + 3 \cdot 1$	71	`cc848a79a96c`
(2) UTF-8, `\r\n`	$66 + 2 + 3 \cdot 2$	74	`73263c16f5fe`
(3) Windows-1252, `\r\n`	$66 + 3 \cdot 2$	72	`44c4c4c420e6`
(4) BOM, UTF-8, `\n`	$3 + 66 + 2 + 3 \cdot 1$	74	`6ebbdf76e0eb`

In (3) nimmt `open` ohne `encoding=` die Kodierung des Systems, unter deutschem Windows `cp1252` (solange der UTF-8-Modus von Python aus ist), und schreibt im Textmodus für jedes `\n` ein `\r\n`. Die Prüfsumme entsteht aus den Bytes, nicht aus der Tabelle, die ein Programm daraus macht: Ein einziges anderes Byte ergibt eine völlig andere Prüfsumme. (2) und (4) haben gleich viele Bytes, aber andere: drei Bytes BOM am Anfang gegen drei `0D` an den Zeilenenden. Nur Variante (1) hat die Form des Importers (UTF-8 ohne BOM, `\n`). Variante (3) hängt zudem vom Rechner ab: Unter macOS und Linux schreibt derselbe Befehl die Bytes von (1). Zwölf Hexadezimalzeichen tragen je 4 Bit, zusammen $12 \cdot 4 =$ 48 Bit, also $2^{48} \approx 2{,}8 \cdot 10^{14}$ Werte. Eine andere Datei trifft zufällig denselben Wert mit der Wahrscheinlichkeit $2^{-48} \approx 3{,}6 \cdot 10^{-15}$; für den Vergleich zweier Ergebnisse reicht das.

Typischer Fehler

In (a) den Codepunkt `F6` direkt als UTF-8-Byte schreiben. Das ist Windows-1252; in UTF-8 stehen Zeichen ab `80` immer als mehrere Bytes mit den festen Bitmustern `110` und `10`.

Typischer Fehler

In (c) `\r\n` als ein Byte zählen oder „ä“ in UTF-8 als ein Byte. Dann kommen für (2) 71 und für (1) 69 Byte heraus.

Bewertung: (a) Herleitung ö und ß 1 P, beide Wörter in UTF-8 1 P, in Windows-1252 mit Anzahl 1 P. (b) Mojibake 1 P, Byte, Position und beide Meldungen 1 P, BOM 1 P. (c) vier Größen 2 P, Begründung der Prüfsummen 1 P, 48 Bit und Wahrscheinlichkeit 1 P.

Lösung zu Aufgabe 3 Ablauf vorhersagen

(a) `finally` läuft, bevor der Rückgabewert oder die Ausnahme beim Aufrufer ankommt. Der Aufrufer fängt `ValueError` und damit auch die Unterklasse `ZeilenFehler`. Ohne `from e` steht in der vierten Zeile `ZeilenFehler NoneType unlesbar: ''`, der Typ der Ursache ist also `NoneType`: `__cause__` bleibt leer, der ursprüngliche Fehler hängt nur noch als `__context__` daran.

```
fertig '2552'
Summe 2553
fertig ''
ZeilenFehler ValueError unlesbar: ''
fertig '66'
Summe 67
```

(b) Der Aufruf von `celsius` führt noch nichts aus, erst `next` startet den Rumpf. Von Hand: $84{,}8 - 32 = 52{,}8$ und $52{,}8 \cdot 5/9 = 29{,}333\ldots$, gerundet `29,3`; ebenso $51 \cdot 5/9 = 28{,}333\ldots$ und $56{,}4 \cdot 5/9 = 31{,}333\ldots$ „nach“ erscheint erst, wenn der nächste Wert angefordert wird. Nach „Ende“ ist der Generator erschöpft, `list(gen)` liefert eine leere Liste ohne Fehler.

```
angelegt
Start
vor 83.0
erster 28.3
nach 83.0
vor 84.8
Schleife 29.3
nach 84.8
vor 88.4
Schleife 31.3
nach 88.4
Ende
Rest []
```

(c) `zaehle` findet `werk` in der umgebenden Funktion und `neu` über `nonlocal`, `gelesen` ist global; das `werk` des Moduls bleibt unberührt:

```
(['Illinois/IL_shift_report_07132026.csv', 'Illinois/liesmich.txt'], 1)
1 Stammwerk 2
```

Ohne `nonlocal neu` macht die Zuweisung `neu += 1` den Namen `neu` lokal in `zaehle`. Beim ersten Namen auf `.csv` liest Python ihn, bevor er einen Wert hat: `UnboundLocalError` („cannot access local variable 'neu' where it is not associated with a value“), Traceback auf stderr, auf stdout nichts, Exit-Code 1.

(d) Beide Voreinstellungen werden einmal ausgewertet, wenn `def` läuft. `stufe` bleibt `WARNING`, obwohl `STUFE` später `DEBUG` ist. Der erste und der dritte Aufruf teilen sich ein Wörterbuch, deshalb `'summenzeile': 2`; der zweite bekommt ein eigenes.

```
('WARNING', {'summenzeile': 1})
('WARNING', {'unplausibel': 1})
('WARNING', {'summenzeile': 2})
```

(e) Der Logger lässt ab INFO durch; `aus` gibt alles weiter, `fehler` (ohne Ziel schreibt ein `StreamHandler` auf stderr) nur ab ERROR:

```
stdout: INFO stammwerk_2026-08-06.csv: 12 Dubletten
        WARNING stammwerk_2026-07-14.csv: 2 ohne Temperatur
        ERROR stammwerk_2026-07-22.csv: Datei ist leer
stderr: nordwerk.importer: stammwerk_2026-07-22.csv: Datei ist leer
```

Die DEBUG-Meldung erscheint nirgends. In `bericht.txt` stehen die drei Zeilen von stdout, im Terminal bleibt die Zeile von stderr. Mit `log.setLevel(logging.WARNING)` fällt auf stdout die INFO-Zeile weg, stderr bleibt gleich. Der Wurzel-Logger hat keinen Handler, deshalb erscheint nichts doppelt.

(f) Gerundet: $(28{,}3 + 29{,}3 + 31{,}3)/3 = 88{,}9/3 = 29{,}633$, also $29{,}6\,°\mathrm{C}$. Genau: $(83{,}0 + 84{,}8 + 88{,}4)/3 = 256{,}2/3 = 85{,}4\,°\mathrm{F}$ und $(85{,}4 - 32) \cdot 5/9 = 53{,}4 \cdot 5/9 = 29{,}667$, also $29{,}7\,°\mathrm{C}$. Die

Umrechnung ist linear,

$$\frac{1}{n}\sum_{i=1}^{n}\frac{5}{9}\,(f_i - 32) = \frac{5}{9}\Big(\frac{1}{n}\sum_{i=1}^{n} f_i - 32\Big),$$

deshalb ist es gleich, ob du erst mittelst oder erst umrechnest. Runden ist nicht linear: Jeder Wert wurde hier um 0,0333 abgerundet, das Mittel also auch, und es rutscht unter die Grenze 29,65. Jeder gerundete Wert weicht höchstens 0,05 ab, das Mittel daraus deshalb auch höchstens $\boxed{0{,}05}$; nach dem letzten Runden kann das eine Stelle ausmachen. Für Hallentemperaturen ist das unerheblich. Wer genaue Mittel braucht, mittelt die ungerundeten Werte und rundet erst das Ergebnis.

Typischer Fehler

In (b) „Start“ vor „angelegt“ schreiben. Der Aufruf einer Generatorfunktion legt nur den Generator an; ihr Rumpf läuft erst beim ersten `next`.

Typischer Fehler

In (e) die INFO-Zeile auf stderr erwarten oder die DEBUG-Zeile auf stdout. Was ein Handler zeigt, begrenzen beide Stufen: erst die des Loggers, dann die des Handlers.

Bewertung: (a) sechs Zeilen 1 P, ohne `from … e` 1 P. (b) Reihenfolge 1 P, Werte und Umrechnung von Hand 1 P. (c) beide Zeilen 1 P, ohne `nonlocal` 1 P. (d) drei Zeilen 1 P. (e) stdout und stderr 1 P, Umleitung und WARNING 1 P. (f) beide Mittel 1 P, Begründung und Grenze 1 P.

Lösung zu Aufgabe 4 Konfiguration, Kommandozeile und Review

(a) uv lädt `.env` nur mit `--env-file` (oder über die Variable `UV_ENV_FILE`); in Fall 1 zählt sie also nicht.

Fall	`eingang`	`ausgang`	`log_level`
1	`daten/roh` (Datei)	`export` (Datei)	`WARNING` (Voreinstellung)
2	`daten/roh` (Datei)	`daten/test` (Umgebung)	`INFO` (Umgebung)
3	`daten/juli` (Argument)	`daten/test` (Umgebung)	`DEBUG` (Argument)
4	`/mnt/werk/roh` (Umgebung)	`ergebnis` (Argument)	`WARNING` (Voreinstellung)
5	`daten/roh` (Voreinstellung)	`daten/aufbereitet` (Voreinstellung)	`WARNING` (Voreinstellung)

In Fall 5 findet `tomllib` keinen Abschnitt `import`, denn TOML unterscheidet Groß- und Kleinschreibung. Der Tippfehler fällt nur bei `ausgang` auf, weil die Voreinstellung für `eingang` zufällig gleich ist: `daten/aufbereitet` statt `export`. Die Form `NAME=wert befehl` aus Fall 4 kennt die PowerShell nicht; dort setzt `$env:NORDWERK_EINGANG = "<Pfad>"` die Variable vorher für alle weiteren Befehle im Fenster, bis `Remove-Item Env:NORDWERK_EINGANG` sie entfernt.

(b)

Nr.	stdout	stderr	Exit-Code
1	`21 Zeilen, 8 Spalten`	leer	0
2	leer	Traceback mit `UnicodeDecodeError` beim Byte `0xfc`	1
3	`12 Zeilen, 1 Spalten`	leer	0
4	leer	nichts sichtbar, die Meldung „ist leer“ wird verworfen	1
5	leer	`usage: …` und „the following arguments are required: datei“	2

(1) Kopf, 20 Schichten (5 Tage, 2 Linien, 2 Schichten) und die Summenzeile ergeben 22 Zeilen; das Programm zählt die Summenzeile mit. Die BOM stört nicht, sie steht nur im ersten Spaltennamen. (2) Die Juli-Datei ist Windows-1252, das „ü“ aus „Früh“ bricht `decode("utf-8")` ab; die Ausnahme ist nicht abgefangen, Python schreibt einen Traceback und endet mit 1. (3) Die September-Datei ist UTF-8, aber mit Semikolon getrennt; ein Komma kommt im Kopf nicht vor. Das Ergebnis ist falsch, ohne Fehlermeldung. (4) `2>/dev/null` verwirft stderr, der Exit-Code bleibt; `uv run` gibt ihn unverändert weiter. In der PowerShell heißt die Umleitung `2>$null`, den Exit-Code zeigt `$LASTEXITCODE`. (5) Ohne `datei` meldet `argparse` den Fehler selbst und endet mit 2.

(c) Zum Beispiel:

Problem	Folge	im Paket `nordwerk`
Schlüssel im Code	steht für immer in der Git-Geschichte, jede Person mit dem Repository kennt ihn	in `.env`, gelesen über `os.environ`
absoluter Windows-Pfad	den Ordner gibt es auf keinem anderen Laptop	`eingang` aus Argument, Umgebung oder `nordwerk.toml`, Pfade mit `pathlib`
Reihenfolge von `glob`	je Dateisystem andere Reihenfolge, andere Ergebnisse	`sorted()` in `lieferungen`
`open` ohne Kodierung	Windows liest `cp1252`, Linux UTF-8, jede Datei anders	Bytes lesen, Kodierung in `lies_text` festlegen
`except: pass`	verschluckt jeden Fehler, danach läuft der Code mit altem oder fehlendem `text` weiter	erwartete Ausnahmen fangen, Datei ablehnen, im Bericht zählen
`print` in Bibliothekscode	Ausgabe lässt sich weder abschalten noch umleiten	`log.info` mit Logger je Modul

(d) (1) Den Ordner `C:\Users\kim\Desktop\roh` gibt es unter Linux nicht, der Rückstrich ist dort sogar ein gewöhnliches Zeichen im Namen. `glob` findet nichts und meldet nichts: Die Funktion liefert `[]`, ausgegeben wird nichts.
(2) Ausgabe `gelesen: daten/roh/stammwerk_2026-09-01.csv` und `gelesen: daten/roh/stammwerk_2026-08-31.csv`. Die Datei vom 1. September ist UTF-8 und ergibt 12 Zeilen. Die vom 31. August ist Windows-1252, `open(name).read()` scheitert, `except: pass` verschluckt das, und `text` enthält noch den 1. September. Die Funktion liefert `24 Zeilen`: die 12 Schichten vom 1. September doppelt, also 12 Dubletten, und keine vom 31. August. Niemand bemerkt es.
(3) Die erste Datei scheitert, `text` hat noch keinen Wert. Nach `gelesen: daten/roh/stammwerk_2026-08-31.csv` bricht die Funktion mit `UnboundLocalError` für `text` ab. Dasselbe Programm liefert je nach Reihenfolge der Dateien ein falsches Ergebnis oder einen Absturz.

Typischer Fehler

In (a) bei Fall 1 `INFO` und `daten/test` eintragen, weil `.env` im Ordner liegt. Die Datei wirkt nur, wenn jemand sie lädt: `uv run --env-file .env` oder der Debugger in VS Code.

Typischer Fehler

In (b) bei Aufruf 3 einen Fehler erwarten. Die Datei ist gültiges UTF-8, nur das Trennzeichen passt nicht; das Programm zählt still falsch. Ein Ergebnis ohne Fehlermeldung ist noch kein richtiges Ergebnis.

Bewertung: (a) Fälle 1 und 2 1 P, Fälle 3 und 4 1 P, Fall 5 1 P. (b) Aufrufe 1 und 3 1 P, Aufrufe 2 und 4 1 P, Aufruf 5 1 P. (c) drei Probleme mit Folge und Lösung 1 P, zwei weitere 1 P. (d) Teil (1) 1 P, Teile (2) und (3) 1 P.